Apache mod_rewrite
the Swiss Army Knife of URL manipulation

"The great thing about mod_rewrite is it gives you all the configurability and flexibility of Sendmail.
The downside to mod_rewrite is that it gives you all the configurability and flexibility of Sendmail."

-- Brian Behlendorf
Apache Group

"Despite the tons of examples and docs, mod_rewrite is voodoo. Damned cool voodoo, but still voodoo. "

-- Brian Moore

These two quotes are the first to meet the eye of someone daring to cast his eyes on the documentation for mod_rewrite. If they are not enough to scare him away, then he will soon meet another statement that should finish the kill:

“But all this functionality and flexibility has its drawback: complexity. So don't expect to understand this entire module in just one day.”

The goal of this presentation is to teach you more than enough to use and abuse mod_rewrite in the future and all of that in slightly less than an hour. First on the agenda is an introduction to the two most used directives in mod_rewrite, next is a number of common directives from Apache along with examples of how they can be implemented with mod_rewrite. Then we return to those directives, to cover some of their additional options and also cover the remaining directives. For the grand finale we turn to some everyday examples of mod_rewrite in action at its best. One thing that there will not be time to cover in detail is regular expressions, as an alternative I suggest reading *perldoc perlre* [1] or Jeffrey Friedels “Mastering Regular Expressions”[2]

1 Also available online at http://www.perldoc.com/perl5.6/pod/perlre.html
2 “Mastering Regular Expressions” 2nd ed. by Jeffrey Fridel – ISBN: 0-596-00289-0 Published by O'Reilly

1. Directives – RewriteRule

Syntax:	**RewriteRule** *Pattern Substitution*
Default:	None
Context:	server config, virtual host, directory, .htaccess
Override:	FileInfo

The RewriteRule directive is the first thing to be checked by mod_rewrite once the RewriteEngine is turned on. Mod_rewrite starts by deciding wether the requsted uri mathches the *Pattern* specified in the RewriteRule, if it does not, then mod_rewrite will do nothing further about that rule. RewriteRules placed in either the server config or in a virtual host will be executed in the *URI translation phase*[3] or if the rules are placed in a .htaccess file, then they will be handled in the *fixup* phase. As is common with all things placed in .htaccess files, the performance is not very good and even more so with RewriteRules because once the rewrite has been made, then Apache will have to rerun the request from the *URI translation phase*.

2. Directives – RewriteCond

Syntax:	**RewriteCond** *TestString CondPattern*
Default:	None
Context:	server config, virtual host, directory, .htaccess
Override:	FileInfo

The RewriteCond directive is used to limit when RewriteRules are executed. If mod_rewrite has already determined that the *Pattern* in the RewriteRule mathches the current URI, then it checks for any RewriteConds placed immediately before the RewriteRule. The RewriteRule will only be executed if the *TestString* successfully mathches the *CondPattern* in the RewriteCond. Multiple RewriteConds placed in succession are joined by an implicit and unless they are followed by an [OR].

3. Imitation

The first set of examples explain mod_rewrite by comparing common directives from other parts of Apache with solutions to the same problems using mod_rewrite.
This is in no way an invitation to use mod_rewrite for those tasks in

3 See Figure 3-3 on page 60 of "Writing Apache Modules with Perl and C" by Lincoln Stein and Doug MacEachern ISBN: 1-56592-567-X Published by O'Reilly

the future, it is only here to serve as examples. All these “imitation” examples work better and faster in the non-mod_rewrite version.

4. **Imitation – Redirect**

The mod_alias version:
Redirect /test/ http://example.com/test/

The mod_rewrite version:
RewriteEngine On
RewriteRule ^/test/(.+) http://example.com/test/$1 [R]

Here we perform the simple task of redirecting everything in a location to an external server. Note that Redirect does prefix matching, which means that we have to do something similar in our RewriteRule. This is accomplished by using (.+) to match any possible remaining part of the url and appending it to the target url by using backreferences ($1). Appending [R] to the RewriteRule forces an external redirect. The redirect can be changed to use another HTTP status code by using [R=code] instead.

5. **Imitation – ProxyPass**

The mod_proxy version
ProxyPass /example/ http://example.com/

The mod_rewrite version
RewriteRule ^/example/(.*) http://example.com/$1 [P]

This ProxyPass emulation is very much the same as the previous example, except that we use the proxy flag [P].

6. **Imitation – UserDir**

The mod_userdir version
UserDir /usr/web -> /usr/web/joe/one/two.html
UserDir /home/*/web -> /home/joe/web/one/two.html

The mod_rewrite version
RewriteRule ^/~(.*)/(.*)$ /usr/web/$1/$2
RewriteRule ^/~([a-z][a-z0-9]+)/(.*)$ /home/$1/web/$2

Now here is a slightly more involved example or actually two

examples. This covers two out of three ways to use UserDir, the third type (UserDir public_html) has been left out because there is no simple solution to it with mod_rewrite. The trick here is to match the username by itself and the rest of the url in another () and use backreferences for both of them when constructing the path. The main difference between the first and the second regular expression is that we use some well known properties of usernames to narrow the mathches to something more valid.

7. Imitation – mod_vhost_alias

The vhost_alias version:
VirtualDocumentRoot /vhosts/%0/docs/

The mod_rewrite version1:

```
RewriteEngine On
RewriteRule          ^(.+)$      /vhosts/%{HTTP_HOST}/docs$1
```

This is our first introduction to environment variables, the standard variables in Apache plus a few extra are available to mod_rewrite in the form %{VARIABLE_NAME}. A full list of the available variables is in the mod_rewrite documentation under RewriteCond [4]

The mod_rewrite version2:

```
RewriteCond          %{HTTP_HOST}     ^(.*)$
RewriteRule          ^(.*)$           /vhosts/%1/docs$1
```

This version is roughly the same as version 1, except here we use a RewriteCond to do the extraction of the server name. Note, that to use a backreference from what was matched in the previous RewriteCond, you have to use %1 instead of $1. The first version is the faster of the two, because mod_rewrite only has to do one pass and only one regular expression.

4 http://httpd.apache.org/docs-2.0/mod/mod_rewrite.html#rewritecond

8. Imitation – trailing slash

Why does accessing directories only work when I include the trailing "/"

ServerName example.com

The mod_rewrite solution:
RewriteCond %{REQUEST_FILENAME} -d
RewriteRule ^(.+[^/])$ $1/ [R]

This seems to be a common problem for many of the people visiting the #apache channels I'm on. The problem is that without the correct ServerName Apache can't redirect to the correct url including the missing /, because it does not have the name necessary to form the url.
Using mod_rewrite, we do two things, with the RewriteRule we make sure that we only try this on url's that does not end with a “/”. The second thing we do (remember that the RewriteConds are not checked before the second round), is to test wether this is a directory and if it is then we go through the RewriteRule to add a “/” to the end of the url. After that we simply issue a Redirect with [R] as before.

9. Directives – RewriteRule 2

RewriteRule *Pattern Substitution [Flags]*

We return to the RewriteRule again to list the flags we have seen in the previous examples along with some new flags that we have not seen before.

[flags]:

'redirect|R [=code]' (force redirect)
'proxy|P' (force proxy)

We have already met these two flags in the examples.

'forbidden|F' (force URL to be forbidden)
'gone|G' (force URL to be gone)
The first sends a 403 HTTP header, while the second sends a 410

'type|T=MIME-type' (force MIME type)

This can be used to change the mime-type of a given request.

'noescape|NE' (no URI escaping of output)

This tells mod_rewrite not to escape special characters like % = and others.

'env|E=VAR:VAL' (set environment variable)

With this we can set an environment variable, and make it available to subsequent parts of the request. Another use could be to pass information between different stages in the rewrite process.

'cookie|CO=NAME:VAL:domain[:lifetime[:path]]' (set cookie)

Note that the cookie feature is only available in Apache 2 (according to the documentation).

10. Directives – RewriteCond 2

RewriteCond *TestString CondPattern [Flags]*

The CondPattern is matched like perl regular expressions with a few extensions:

Lexical (string) compares can be made with (< > =).

'-d' (is directory)

Matches if *TestString* is a directory.

'-f' (is regular file)
'-l' (is symbolic link)

As above, but for files and symbolic links.

'-s' (is regular file with size > 0)

With this the check is like for **'-f'** but with the added condition, that the file has a size greater than 0 bytes.

11. Directives – RewriteOptions

RewriteOptions option (default Maxredirects=10)

This directive has two options – Maxredirects and inherit. Inherit lets rewriterules be inherited from the parent. Maxredirects is a new option which has been added recently to protect against loops. A very common mistake in mod_rewrite is to rewrite an url, such that the target also will be picked up by the same RewriteRule. Often such mistakes can be fixed by adding a RewriteCond %{REQUEST_URI} !*target*. Maxredirects will not repair such mistakes, but it will put an entry in the error log instead of causing a newerending loop.

12. Directives – Other

RewriteEngine on|off (default off)

Should we turn mod_rewrite on?

RewriteLog file
RewriteLogLevel Level (default 0)

Here we can define a file where mod_rewrite can log what it is doing. RewriteLogLevel will make mod_rewrite more and more verbose as the level is increased from 1 – 9, 0 means no logging and above 9 will be the same as 9. This degrades performance and should not be used in production.

RewriteOptions Option (inherit)

This directive can be used in a virtual host to make it inherit all RewriteRules from the Server context. *Option* can only be set to inherit. A convenient way to define often used rules in only one place.

RewriteBase URL-path (context: dir, .htaccess)

The RewriteBase directive explicitly sets the base URL for per-directory rewrites. There it will act locally, i.e., the local directory prefix is stripped at this stage of processing and your rewriting rules act only on the remainder. At the end it is automatically added back to the path.

12. Directives – RewriteMap

Syntax: RewriteMap *MapName MapType:MapSource*
Default: not used per default
Context: server config, virtual host

RewriteMap defines a mapping function for key lookups. NB! RewriteMaps can be used in a .htaccess, but it cannot be defined there.
The *MapType* can be one of:

int (toupper,tolower,escape,unescape)
The internal map has for predefined functions, we will see examples of their use later.

prg (external program)

This lets you pass of the map to an external program. See also RewriteLock.

txt

A simple lookup of key/value pairs in a text file.

rnd

This is like txt but makes a random choice from | seperrated lists.

dbm[=type] (one of gdbm, sdbm, ndbm or db)

With dbm you can keep your maps in a file system database. The choice of types depends on which db libraries Apache has been linked with, you can chek valid types with the command bin/apxs -q AP_LIBS and looking for the libraries there or if Apache has been dynamically linked with the db libraries, then you can use the command ldd bin/httpd.

13. Examples

This group of examples differ from the previous by being things that should be solved by mod_rewrite. In most cases there are no obvious alternatives.

14. Example – No deep links / disallow external referers

This example is was inspired by some of the recent debates about

“deep linking” and other ways of “borrowing” content.

```
RewriteCond %{HTTP_REFERER} !=""
RewriteCond %{HTTP_REFERER} !^http://example.com/.*$ [NC]
RewriteRule .*        -             [F]
```

Here we use the RewriteRule to match everything, and return forbidden [F] if we get there. The first RewriteCond allows browsers that do not have the HTTP_REFERER set, because that is usually a sign of accessing the site directly instead of through a link. The second RewriteCond only matches if the HTTP_REFERER does not include our own base url.

```
Disallow UserAgent
RewriteCond %{HTTP_USER_AGENT} EvilBot.*
RewriteRule /nobot/.*          http://127.0.0.1/      [R=301]
```

Here we use a similar strategy to look for user agents accessing the /nobot/ area of our site, but instead of sending a forbidden header, we redirect them permanently to their own host. Redirecting the bot back to itself is a bit silly, but in the very rare case that someone has a website on their EvilBot, it may puzzle the admin.

15. Example - How to restructure homedirs

This is one I ran into a couple of years ago when we were giving away user pages to all the dial up customers who checked the right box during registration. All went well for a while, up to the point when we hit close to 32k users. Solaris stopped playing nice because it did not like having more than 32k links to one file (..) The first choice would be to simply change the path of their homedir in the password file, but that was not an option because we only have virtual users with no actual homedir (some people may also insist that the actual reason for using subdirectories instead of a regular ~user setup was a decision by marketing made because they could not find the ~ button on their keyboards)

```
RewriteEngine on
RewriteRule          ^/(([a-z])[a-z0-9]+)/(.*) /data/$2/$1/$3
```

The first solution we came up with was to grab the first letter of the username and then place each username beginnig with a in a directory named a, and so on.

```
RewriteMap    LC    int:tolower
RewriteRule         ^/(([a-z])[a-z0-9]+)/(.*)
                    /data/${LC:$2}/${LC:$1}/$3 [NC]
```

The second example uses the same structure as the first, but it adds the extra feature of lowercasing the usernames. This is done by defining an internal RewriteMap LC using the function tolower. After that, the usernames can simply be lowercased by using ${mapname:variable} or in this case ${LC:$1}.

16. Example - How to redirect a logged in user to a specific page

This was a question asked in #apache on opn[5] not too long ago.

```
RewriteEngine on
RewriteMap    user-to-page   txt:/path/to/map.user-to-page
RewriteCond   %{LA-U:REMOTE_USER}  ^(.+)$
RewriteRule   ^/$  ${user-to-page:%1|/notfound.html}
```

First we define a txt RewriteMap called user-to-page like this:

```
/path/to/map.user-to-page
user1 /some/where/user1.html
user2 /some/where/else/user2.html
...
```

Each user maps to some internal location. The RewriteRule is like we have seen before, rewriting requests for / to what it can find in the map or if for some reason the user is not found, then it shows the notfound.html page instead. The RewriteCond introduces a new map like functionality for looking up the REMOTE_USER, which will fire a subrequest to determine the username, which is not available yet, because we have not yet reached the API stage for user authentication yet.

17. Example - Webmail redirector

Webmail redirector is just one example of how this type of rule can be used to redirect based on part of the domain name.

```
RewriteEngine on
RewriteCond %{HTTP_HOST}  ^webmail\.(.*)\.dk$          [NC]
RewriteRule ^/$      http://mail.dk/login.x?dom=%1     [R,NE]
```

5 irc.openprojects.net – see also http://freenode.info/

Nothing really new and exciting going on here, except that we tell RewriteCond to do a case insensitive match, similar to //i in perl regular expressions. We also use part of the domain name to form the query string and to avoid the = sign being escaped, we use no-escape, NE, along with the forced redirect, R.

18. **Example –** moving from index files to autoindex

This was another question asked on opn[6]. He used to have a manually generated index file in his download directory, and many people had their bookmarks pointing directly to that file, such that now when he had changed to mod_autoindex, his error log was being filled by 404's from people using their old bookmarks. His own first attempt was to use Redirect to redirect all requests to index.html back to /, but all that got him was a loop. In this case we use a .htaccess file.

/download/.htaccess

```
RewriteEngine on
RewriteBase /download/
RewriteCond     %{REQUEST_FILENAME}     !-d
RewriteRule     ^(.*)/index\.html$      $1/     [R=301]
```

Because this is used in a .htaccess file, we set RewriteBase. The RewriteRule checks to see if we are requesting a file named index.html. The RewriteCond make sure that we only do this if it is not a directory (you could argue that directories are usually not named index.html/). Matching and collecting any prepended directories in the RewriteRule makes this example work in subdirectories also. Adding the following RewriteCond before the one that is already there could bring additional benefits:

```
RewriteCond     %{REQUEST_FILENAME}     !-f
```

By checking that the file is not there, we make sure that if at some point in the future we chose to put an index.html there again, then it would not be affected by this rule.

6 irc.openprojects.net – see also http://freenode.info/ - thanks to Morbus for asking the question.

The latest version of this presentation is available at
http://www.toftum.org/slides/ApacheCon/

Questions, etc.

mads@apache.dk

This presenation has ended.
Click anywhere to return.

_

